

ASSESSMENT REPORT

PYTHON AUTOMATION WIPRO

TEAM MEMBERS:

RUMANI SADHUKHAN-12023052019035

Part B – Technical Investigation Questions

Q1. What is Selenium, and which types of applications is it designed to automate? State two suitable and two unsuitable use cases.

ANS: Selenium is an open-source automation framework used to automate web browsers. It allows us to write programs that open a browser, navigate to websites, click buttons, enter data, submit forms, and verify web-page behavior.

Selenium is mainly designed for web applications running in browsers such as Chrome, Firefox, Edge, and Safari.

Suitable use cases:

1. Automated testing of an e-commerce website — checking login, product search, cart, and checkout.
2. Testing a web-based banking application — checking whether login, account pages, and transactions work correctly in a test environment.

Unsuitable use cases:

1. Testing desktop applications such as Microsoft Word because Selenium is designed for web browsers.
2. Testing mobile-native applications directly, such as Android/iOS apps. Tools such as Appium are more appropriate for native mobile automation.

```
C:\Users\Shreyanka>pip show selenium
Name: selenium
Version: 4.47.0
Summary: Official Python bindings for Selenium WebDriver
Home-page: https://www.selenium.dev
Author:
Author-email:
License: Apache-2.0
Location: C:\Users\Shreyanka\AppData\Roaming\Python\Python311\site-packages
Requires: certifi, trio, trio-websocket, typing_extensions, urllib3, websocket-client
Required-by:
```

Q2. Differentiate Selenium WebDriver, Selenium IDE, and Selenium Grid.

Feature	WebDriver	Selenium IDE	Selenium Grid
Purpose	Programmatically automate browsers	Record and replay browser actions	Run tests on multiple environments
Coding	Requires programming	Little/no coding required	Usually works with WebDriver tests
Main use	Automated testing	Quick test creation/prototyping	Parallel and cross-browser testing

Feature	WebDriver	Selenium IDE	Selenium Grid
Example	Python/Java Selenium script	Record clicks and replay them	Run Chrome, Firefox and Edge tests simultaneously

Q3. Explain the roles of the test script, Selenium client library, browser driver, browser, and WebDriver protocol during one command such as `driver.get()`.

ANS: 1. Test script

This is our Python/Java/etc. program.

Example:

```
driver.get("https://example.com")
```

It tells Selenium what action we want to perform.

2. Selenium Client Library

The Selenium library provides programming-language APIs.

For Python:

```
from selenium import webdriver
```

It converts our Python command into a WebDriver command.

3. WebDriver Protocol

The command is communicated using the W3C WebDriver standard, which defines how automation commands are sent to browsers and how responses are returned.

4. Browser Driver

The browser driver acts as an intermediary between Selenium and the browser.

For example:

Chrome → ChromeDriver

Firefox → GeckoDriver

Edge → EdgeDriver

5. Browser

The browser finally performs the requested action — in this case, navigating to the specified URL.

Q4. Why is a virtual environment recommended? What problem can occur when packages are installed globally?

ANS: A virtual environment creates an isolated Python environment for a particular project.

For example:

```
python -m venv myenv
```

Then we can install Selenium inside it:

```
pip install selenium
```

For example:

Project A → Selenium version X

Project B → Selenium version Y

A virtual environment allows both projects to have their own dependencies without interfering with each other.

Problem with global installation

When packages are installed globally, one project's package upgrade or downgrade can affect another project.

This can cause:

- Dependency conflicts

- Version incompatibility

- Broken projects

- Difficult-to-reproduce errors

```
C:\Users\Shreyanka>py -3.14 -m venv selenium_env
C:\Users\Shreyanka>selenium_env\Scripts\activate
(selenium_env) C:\Users\Shreyanka>python --version
Python 3.14.7
(selenium_env) C:\Users\Shreyanka>pip --version
pip 26.2.1 from C:\Users\Shreyanka\selenium_env\Lib\site-packages\pip (python 3.14)
```

Q5. What is Selenium Manager? How has it changed driver setup in recent Selenium versions?

ANS: Selenium Manager is a tool included with modern Selenium releases that helps automatically manage browser drivers.

Previously, users often had to:

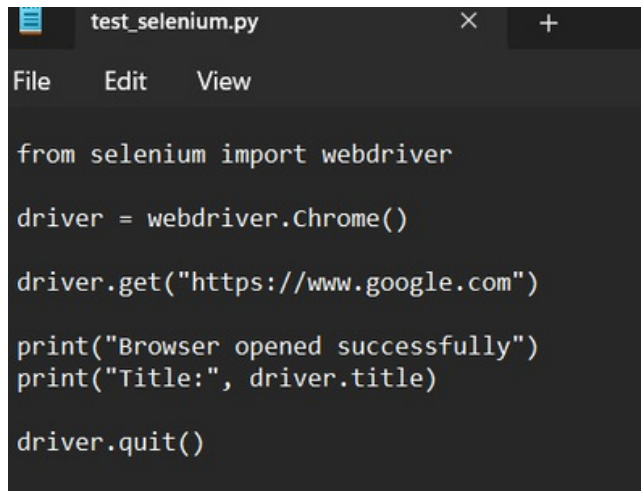
1. Download the correct browser driver manually.
2. Find the correct driver version.
3. Add the driver to PATH or specify its location.

For example, older setups commonly required manually downloading ChromeDriver.

Selenium Manager can detect the browser and help obtain/use a compatible driver when needed.

Advantage

It reduces manual driver setup and helps avoid common browser-driver version problems.



```
test_selenium.py
File Edit View

from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://www.google.com")

print("Browser opened successfully")
print("Title:", driver.title)

driver.quit()
```

Q6. Differentiate driver.find_element() and driver.find_elements(). What is returned when no element matches?

ANS:

Method	Result	No match
find_element()	Single WebElement	NoSuchElementException
find_elements()	List of WebElements	Empty list []

Part C — Installation Challenges to Solve

C1 — pip is not recognized

Problem:

The terminal shows that pip is not recognized or the command cannot be found.

Root cause:

Python or its Scripts directory may not be correctly added to the system PATH, or the terminal may be using a different Python environment.

Solution:

```
(.venv) C:\Users\Asus\selenium_assignment>python test_selenium.py
Page title: Example Domain
Current URL: https://example.com/

(.venv) C:\Users\Asus\selenium_assignment>where python
C:\Users\Asus\selenium_assignment\.venv\Scripts\python.exe
C:\Users\Asus\AppData\Local\Microsoft\WindowsApps\python.exe
C:\Users\Asus\AppData\Local\Python\bin\python.exe

(.venv) C:\Users\Asus\selenium_assignment>where pip
C:\Users\Asus\selenium_assignment\.venv\Scripts\pip.exe
C:\Users\Asus\AppData\Local\Python\bin\pip.exe

(.venv) C:\Users\Asus\selenium_assignment>python -m pip --version
pip 26.1.2 from C:\Users\Asus\selenium_assignment\.venv\Lib\site-packages\pip (python 3.14)
```

C2 — Package installed, import fails

Problem:

pip reports Selenium as installed, but Python gives:

```
File "C:\Users\Asus\selenium_assignment\c2_test.py",
  import selenium
ModuleNotFoundError: No module named 'selenium'
```

Root cause:

Selenium was installed in one Python environment, while the IDE or script is using a different Python interpreter.

Solution::

```
(c2_test) C:\Users\Asus\selenium_assignment>python -c "import sys; print(sys.executable)"
C:\Users\Asus\selenium_assignment\c2_test\Scripts\python.exe

(c2_test) C:\Users\Asus\selenium_assignment>python -m pip show selenium
WARNING: Package(s) not found: selenium

(c2_test) C:\Users\Asus\selenium_assignment>python -m pip install selenium
```

C3 — Driver/browser mismatch

Problem:

Selenium fails to create a browser session with an error such as `SessionNotCreatedException`, indicating that the driver supports a different browser version.

Root cause:

The `ChromeDriver` version may not match the installed Chrome browser. A stale manually downloaded driver, a driver on `PATH`, or an outdated cached driver can also cause the problem.

```
(c2_test) C:\Users\Asus\selenium_assignment>where chromedriver
INFO: Could not find files for the given pattern(s).

(c2_test) C:\Users\Asus\selenium_assignment>where chrome
INFO: Could not find files for the given pattern(s).

(c2_test) C:\Users\Asus\selenium_assignment>chrome --version
'chrome' is not recognized as an internal or external command,
operable program or batch file.

(c2_test) C:\Users\Asus\selenium_assignment>set SE_DEBUG=true
```

Result:

Once the browser and driver versions are compatible, `webdriver.Chrome()` can create the browser session successfully.

C4 — Browser binary not found

Problem:

Selenium reports that it cannot find the Chrome browser executable.

Root cause:

The browser is not installed, or Selenium has been given an incorrect/non-standard browser executable path.

Solution:

```
from selenium import webdriver

driver = webdriver.Chrome()

try:
    driver.get("https://example.com")
    print(driver.title)
finally:
    driver.quit()
```

C9 — Element not found

Problem:

The webpage loads, but Selenium throws:

`NoSuchElementException`

Root cause:

The locator may be incorrect, non-unique, or the element may not yet be available. Other possibilities include an iframe, dynamic content, shadow DOM, or changing element IDs.

Solution:

Use a valid locator, for example:

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("https://www.selenium.dev/selenium/web/web-form.html")

    text_box = driver.find_element(By.NAME, "my-text")
    text_box.send_keys("Umani")

    print("Element found and interaction successful.")
finally:
    driver.quit()
```